AI TOPICS COURSE · TEAM PROJECT GUIDE · DOCUMENT 0 OF 5

Regulatory & Support Copilot

The master guide for the whole project. Read this first, then read your own part guide.

Part	Title	Owner (write your name)	Guide
1	Mock Data and Tools		01-Part1-Mock-Data-and-Tools.pdf
2	Knowledge Base and AI Foundation		02-Part2-Knowledge-Base-and-AI-Foundation.pdf
3	Regulation Agents (Workflow A)		03-Part3-Regulation-Agents.pdf
4	Support Agent and Orchestrator (Workflow B)		04-Part4-Support-Agent-and-Orchestrator.pdf
5	Website, Safety and Evaluation		05-Part5-Website-Safety-and-Evaluation.pdf

Contents

1. How to use these documents
2. The project in one page
3. What the finished prototype does
4. System architecture
5. The five parts at a glance
6. Technology stack
7. Repository structure
8. Shared contracts
9. Data policy: mock and AI-generated data
10. Environment setup (every member)
11. GitHub workflow
12. Building your part with your own AI tool
13. Timeline
14. Integration and definition of done
15. Knowing each other's work
16. Final deliverables
17. Glossary

1. How to use these documents

There are six guides. This one describes the whole project and the rules everyone follows. The other five describe one part each, in enough detail that the owner can build it step by step.

- **Everyone reads this whole guide** before choosing a part. You will be asked about all parts in the final presentation, not only yours.
- **After choosing, read your part guide twice**, then skim the other four so you know what your teammates build and what they expect from you.
- **The shared contracts in Section 8 are the law.** If your code follows them, the five parts will fit together. If you need to change a contract, open a pull request and get approval from the whole team (Section 11).
- **When something is unclear**, open a GitHub Issue with the label `question` instead of guessing. A two-line question saves a week of rework.

> **Guide's note:** The biggest risk in a 5-person project is not hard code. It is five people building five pieces that don't connect. Every rule in this guide exists to stop that.

2. The project in one page

Case study. ProgressSoft is a Jordanian fintech company in Amman. It builds payment and clearing systems (cheque clearing, ACH, instant payments, mobile wallets) for central and commercial banks in many countries. We use it as a realistic case study. We have no access to its internal data, so everything we build runs on **mock data** that we create ourselves.

Problem 1: new regulations. Central banks keep publishing new rules and message-format changes (for example, ISO 20022 field changes). Staff read each document by hand, find which product modules and clients are affected, and write change requests. This takes days and depends on a few experts.

Problem 2: support tickets. Banks send tickets such as "Why was payment 7731 rejected?". An engineer must search the transaction logs to find the reason code and explain it. This is slow and pulls engineers away from development.

Our solution. A team of AI agents that prepares the work, and a website where a person reviews it:

- **Workflow A (regulation):** Regulation Watcher summarizes a circular → Impact Analysis finds affected modules and clients → Change Request agent drafts change requests → a human approves, edits or rejects them.
- **Workflow B (ticket):** Support Triage agent reads a ticket → searches the logs → finds the rejection reason → drafts a reply → a human approves it before it is "sent".

What we deliver. A working **prototype**: a website that runs both workflows end to end on mock data, shows every agent step live, shows the sources behind every answer, and keeps a full audit log. Plus an evaluation report that measures how accurate the agents are.

What we do not build. No real bank connections, no real payments, no real customer data, no production security (login is a simple name picker).

3. What the finished prototype does

This is the demo we will give at the end. Build toward it from week 1. If a feature does not serve this demo, it is optional.

Demo A: a new regulation arrives

1. Open the website and go to **New Regulation**.
2. Choose the sample circular `REG-003` from the dropdown (or paste a new one) and click **Analyze**.
3. The **Task page** opens and shows the three agent steps running one after another, each with a status (waiting, running, done, failed) and time taken.
4. When done, the page shows:
    - the **summary**: each change, its effective date, and the exact sentence of the circular it came from;
    - the **impact report**: affected modules, affected clients, severity, and the product-document passages used as evidence;
    - the **draft change requests** with acceptance criteria and test cases.
5. The reviewer picks their name, then clicks **Approve**, **Edit** or **Reject** on each change request.
6. Approved change requests appear on the **Change Requests** page (our mock ticketing system).
7. The **Audit Log** page shows every agent step and every human decision with timestamps.

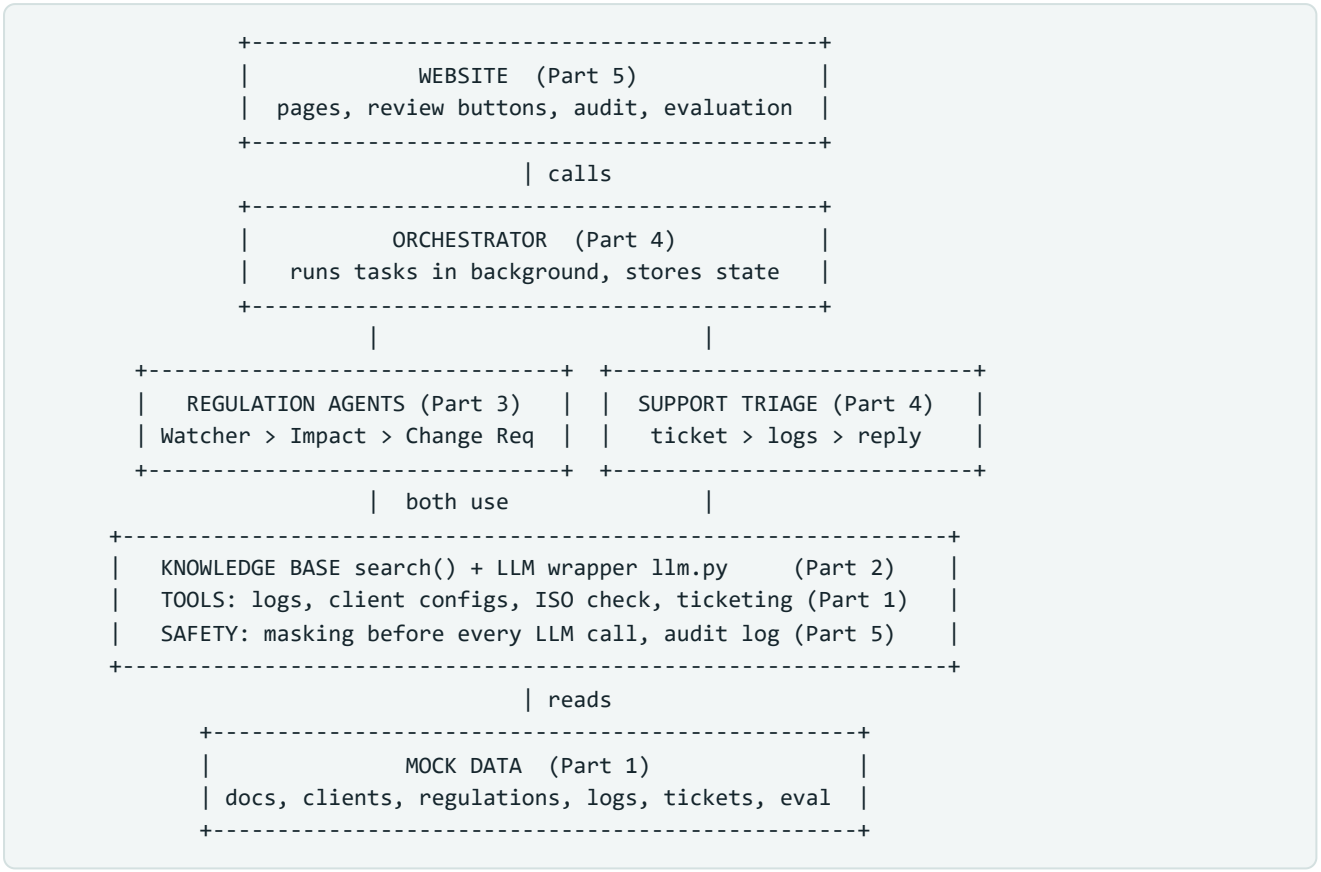
Demo B: a bank opens a ticket

1. Go to **Tickets**, open ticket `TCK-0007` ("Why was payment 7731 rejected?") and click **Run triage**.
2. The Task page shows the triage steps: reading the ticket, searching logs, searching the knowledge base, drafting a reply.
3. The result shows the reason code (for example `AC04`, closed account), the log lines used as evidence, the knowledge-base sources, a confidence level, and a draft reply.
4. The reviewer approves (or edits) the reply. The ticket status changes to "Answered".

Extra pages

- **Knowledge Search:** type a question and see which document chunks the system finds. This shows the class how RAG works.
- **Evaluation:** shows the latest accuracy numbers produced by the evaluation script.

4. System architecture



How a request flows (Workflow A): the website calls `start_regulation_task()` in the orchestrator → the orchestrator creates a task, then calls the three regulation agents in order → each agent uses `search()` for evidence, `llm.complete_json()` to think, and Part 1 tools for facts → the orchestrator saves each output and writes audit events → the website polls the task every 2 seconds and shows progress → the human decision is saved and audited.

5. The five parts at a glance

Part	Folder	Delivers	Needs from	Gives to
1 Mock Data and Tools	`part1_data/`	All mock data, 4 read-only tools, evaluation answer sets	—	Everyone
2 Knowledge Base and AI Foundation	`part2_knowledge/`	Document ingestion, `search()`, the LLM wrapper `llm.py`, config	Part 1 data	Parts 3, 4, 5
3 Regulation Agents	`part3_regulation/`	Watcher, Impact Analysis, Change Request agents	Parts 1, 2	Part 4 (orchestrator)
4 Support Agent and Orchestrator	`part4_support/`	Support Triage agent, task runner, task store	Parts 1, 2, 3, 5 (audit)	Part 5 (website)

Part	Folder	Delivers	Needs from	Gives to
5 Website, Safety and Evaluation	`part5_web/`	Website, masking, audit log, evaluation script and report	Parts 1, 2, 4	The demo

Review pairs (who reviews whose pull requests): Part 1 → reviewed by Part 2 · Part 2 → by Part 3 · Part 3 → by Part 4 · Part 4 → by Part 5 · Part 5 → by Part 1. Changes to `contracts/` need approval from **all** members.

6. Technology stack

These choices are fixed so that the parts fit together. Don't swap a library without a team decision recorded in an Issue.

Area	Choice	Why
Language	Python 3.11 or newer	One language for everything; every AI coding tool knows it well
Data models	Pydantic v2	Shared contracts with automatic validation
Web	FastAPI + Jinja2 templates + plain JavaScript	Simple, no front-end build step, easy to run
Database	SQLite (one file, `runtime/app.db`)	No server to install
Vector database	Chroma (local, persistent folder `runtime/chroma/`)	Pip install, free, runs offline
Embeddings	Chroma's default embedding model (all-MiniLM-L6-v2)	Free and local; downloads about 80 MB on first use
LLM	One wrapper `part2_knowledge/common/llm.py` with providers: `fake`, `ollama`, `anthropic`, `openai`	Everyone can develop free with `fake` or `ollama`; the demo can use a stronger model
Agents	Plain Python functions (no agent framework)	Easy to read, test and explain; fewer surprises
Tests	pytest	Standard and simple
Collaboration	GitHub: Issues, Projects board, pull requests	One place for code, tasks and reviews

About the LLM providers. `fake` returns the example object from the contract, so code runs with no model at all (use it for tests and early integration). `ollama` runs a free local model such as `qwen2.5:7b` or `llama3.1:8b` (needs about 8 GB of RAM). `anthropic` or `openai` use a paid API key; a small, cheap model is enough. The team chooses one provider for the final demo and puts it in `.env`.

7. Repository structure

The repository has **one top-level folder per part**. Each part folder belongs to exactly one owner, who is the only person who edits it. Only `contracts/` and `tests/integration/` are shared. Empty folders hold a `.gitkeep` file, and every Python folder has an `__init__.py` so parts can import each other (for example `from part1_data.tools.log_search import search_logs`).

```
regulatory-copilot/
├── README.md                    how to install and run (Part 5 keeps it updated)
├── requirements.txt             all Python packages (add yours by pull request)
├── .env.example                 every setting, with no secret values
├── .gitignore                   ignores .env, .venv/, runtime/, __pycache__/
├── .github/
│   ├── CODEOWNERS               maps each part folder to its owner
│   └── pull_request_template.md
├── contracts/                   SHARED: schemas.py + CONTRACTS.md (all approve changes)
│
├── part1_data/                  PART 1 – Mock Data and Tools
│   ├── data/                    company/, clients/, regulations/, logs/, tickets/, history/,
eval/, generators/
│   ├── tools/                   regulations.py, log_search.py, client_config.py,
iso_validator.py, ticketing.py
│   └── tests/
├── part2_knowledge/             PART 2 – Knowledge Base and AI Foundation
│   ├── knowledge_base/          ingest.py, search.py
│   ├── common/                  config.py, llm.py
│   └── tests/
├── part3_regulation/            PART 3 – Regulation Agents
│   ├── watcher.py, impact.py, change_request.py, cli.py
│   ├── prompts/
│   └── tests/
├── part4_support/               PART 4 – Support Agent and Orchestrator
│   ├── triage.py
│   ├── prompts/
│   ├── orchestrator/            orchestrator.py, store.py, run.py
│   └── tests/
├── part5_web/                   PART 5 – Website, Safety and Evaluation
│   ├── app.py, templates/, static/
│   ├── safety/                  masking.py, audit.py, guidelines.md
│   ├── evaluation/              run_eval.py
│   └── tests/
│
├── tests/integration/           SHARED: end-to-end tests (Parts 4 and 5)
├── docs/                        these guides + the final report
├── presentation/                animated explainer
└── runtime/                     created at run time (database, Chroma); never committed
```

Running code: always run commands from the repository root, for example `python -m part2_knowledge.knowledge_base.ingest` or `uvicorn part5_web.app:app --reload`. Each part folder has its own `README.md` and `AI_LOG.md`.

8. Shared contracts

Contracts are the data shapes and function signatures that connect the parts. They live in `contracts/schemas.py` as Pydantic models. Each model must include one realistic example (in `model_config = {"json_schema_extra": {"example": {...}}}`), because the `fake` LLM provider returns that example and the website uses it for previews.

Contract Day (end of week 1): each owner writes the models for their part (listed below) in one pull request, and everyone reviews all of them together in a 1-hour meeting. After that, changes need everyone's approval.

8.1 Data models

Model	Written by	Key fields
`RegulationDoc`	Part 1	`regulation_id`, `title`, `issuer`, `country`, `published` (date), `effective` (date or null), `text`
`Client`	Part 1	`client_id`, `name`, `type` (central / commercial / wallet), `country`, `modules` (list), `message_standard`, `local_rules` (list)
`LogEntry`	Part 1	`timestamp`, `level` (INFO / WARN / RJCT / ERROR), `message_type`, `ref`, `client_id`, `status`, `reason_code` (or null), `detail`
`Ticket`	Part 1	`ticket_id`, `client_id`, `created_at`, `subject`, `body`, `priority`, `status` (open / answered), `transaction_ref` (or null)
`SearchResult`	Part 2	`text`, `source` (file path), `doc_type`, `location` (section title), `score` (0–1), `metadata` (dict)
`Change`	Part 3	`change_id`, `description`, `category` (message_format / reporting / aml / operational / other), `mandatory`, `source_quote`, `source_location`
`RegulationSummary`	Part 3	`regulation_id`, `title`, `country`, `effective_date`, `summary_text`, `changes` (list of `Change`)
`Impact`	Part 3	`change_id`, `module`, `clients` (list of ids), `severity` (high / medium / low), `reason`, `sources` (list of `SearchResult`)
`ImpactReport`	Part 3	`regulation_id`, `impacts` (list of `Impact`), `unmatched_changes` (list of change ids)
`TestCase`	Part 3	`test_id`, `steps` (list), `expected`
`ChangeRequest`	Part 3	`cr_id`, `regulation_id`, `title`, `description`, `module`, `clients`, `acceptance_criteria` (list), `test_cases` (list), `sources`, `status` (pending_review / approved / rejected)
`TriageResult`	Part 4	`ticket_id`, `transaction_ref`, `reason_code`, `root_cause`, `evidence` (list of `LogEntry`), `sources` (list of `SearchResult`), `draft_reply`, `confidence` (high / medium / low), `escalate` (bool)

Model	Written by	Key fields
`TaskStep`	Part 4	`name`, `status` (waiting / running / done / failed), `started_at`, `finished_at`, `error`
`Task`	Part 4	`task_id`, `type` (regulation / ticket), `status` (running / pending_review / approved / rejected / failed), `input_ref`, `steps` (list of `TaskStep`), `outputs` (dict), `created_at`
`ReviewDecision`	Part 5	`task_id`, `item_id`, `decision` (approve / edit / reject), `edited_content` (or null), `reviewer`, `comment`, `timestamp`
`AuditEvent`	Part 5	`timestamp`, `actor` (agent name or reviewer), `action`, `task_id`, `details` (dict)

Example of one model in JSON form, so you see the level of detail expected:

```
{
  "ticket_id": "TCK-0007",
  "transaction_ref": "7731",
  "reason_code": "AC04",
  "root_cause": "The receiving account at BANK-C is closed.",
  "evidence": [{"timestamp": "2026-03-02T09:12:03", "level": "RJCT", "message_type": "pacs.008",
                "ref": "7731", "client_id": "BANK-C", "status": "rejected",
                "reason_code": "AC04", "detail": "Creditor account closed"}],
  "sources": [{"text": "AC04 – Closed account number ...", "source":
"part1_data/data/company/reason_codes.md",
               "doc_type": "reason_codes", "location": "AC04", "score": 0.82, "metadata": {}}],
  "draft_reply": "Dear BANK-C team, payment 7731 was rejected because the receiving account is
closed ...",
  "confidence": "high",
  "escalate": false
}
```

8.2 Function interfaces

Each part exposes these functions. Other parts call **only** these, never internal helpers.

Owner	Function	Returns
Part 1	`part1_data.tools.log_search.search_logs(ref=None, client_id=None, level=None, since=None, until=None)`	`list[LogEntry]`
Part 1	`part1_data.tools.client_config.list_clients()` / `get_client(client_id)`	`list[Client]` / `Client`
Part 1	`part1_data.tools.iso_validator.validate_message(xml_text)`	`{"valid": bool, "errors": list[str]}`
Part 1	`part1_data.tools.ticketing.list_tickets()` / `get_ticket(id)` / `set_ticket_status(id, status)`	`Ticket` objects

Owner	Function	Returns
Part 1	`part1_data.tools.ticketing.create_change_request(cr)` / `list_change_requests()`	id / `list[ChangeRequest]`
Part 1	`part1_data.tools.regulations.list_regulations()` / `get_regulation(id)` / `parse_regulation(text)`	`RegulationDoc` objects
Part 2	`part2_knowledge.knowledge_base.search.search(query, k=5, filters=None)`	`list[SearchResult]`
Part 2	`part2_knowledge.knowledge_base.ingest.ingest_all()`	stats dict (files, chunks)
Part 2	`part2_knowledge.common.llm.complete_text(system, user)` / `complete_json(system, user, schema)`	`str` / instance of `schema`
Part 3	`part3_regulation.watcher.summarize(doc)`	`RegulationSummary`
Part 3	`part3_regulation.impact.analyze(summary)`	`ImpactReport`
Part 3	`part3_regulation.change_request.draft(summary, report)`	`list[ChangeRequest]`
Part 4	`part4_support.triage.triage(ticket)`	`TriageResult`
Part 4	`part4_support.orchestrator.start_regulation_task(doc)` / `start_ticket_task(ticket)`	`task_id` (runs in background)
Part 4	`part4_support.orchestrator.get_task(id)` / `list_tasks()` / `set_task_status(id, status)` / `update_output(id, key, value)` / `retry_task(id)`	`Task` objects
Part 4	`part4_support.orchestrator.run_regulation_sync(doc)` / `run_ticket_sync(ticket)` (no background thread, for evaluation)	finished `Task`
Part 5	`part5_web.safety.masking.mask(text)`	masked `str`
Part 5	`part5_web.safety.audit.log_event(actor, action, task_id, details)` / `list_events(task_id=None)`	`None` / `list[AuditEvent]`

Stubs rule. Until a function is ready, its owner merges a **stub**: the real signature that returns the contract example (or, for `mask`, the text unchanged). This lets everyone else build and test from week 2 without waiting.

9. Data policy: mock and AI-generated data

Real data from ProgressSoft or any bank is not available and must not be used. Part 1 creates a complete fictional world:

- **Fictional names only.** Clients are `BANK-A` to `BANK-E` with invented names (for example "Bank of Northfield"). Regulators are invented ("Central Bank of Country X"). Never use a real bank's name, logo or document.
- **AI-generated text is allowed and expected** for circulars, product documents and tickets. A human must check every generated file for consistency (Part 1 guide, Section 6).

- **Logs are generated by a script with a fixed random seed**, so every member gets identical data.
- **Real public standards are fine to use**: ISO 20022 message names (pacs.008, pacs.002) and reason codes (AC01, AC04, AM04 …) are public and make the prototype realistic.
- **Mark it.** Every data file starts with the line `MOCK DATA – fictional, for a student prototype`.
- **Fake personal data for safety tests** (an IBAN-shaped number, a phone number) must be clearly invented.

10. Environment setup (every member)

Do this in week 1, before writing any code. Ask for help in the team chat if a step fails.

1. **Install the tools:** Git, Python 3.11+, VS Code (or your editor), and your AI coding tool. Optional: Ollama (free local LLM) from ollama.com, then run `ollama pull qwen2.5:7b`.
2. **Create a GitHub account** (if you don't have one) and send your username to the team lead.
3. **Tell Git who you are** (one time):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

4. **Accept the invitation** to the repository (check your email), then clone it:

```
git clone https://github.com/<team-lead>/regulatory-copilot.git
cd regulatory-copilot
```

5. **Create a virtual environment and install packages:**

```
python -m venv .venv
.venv\Scripts\activate          (Windows)
source .venv/bin/activate       (macOS / Linux)
pip install -r requirements.txt
```

6. **Create your settings file:** copy `.env.example` to `.env` and fill it. Start with `LLM_PROVIDER=fake`. Never commit `.env`.

```
LLM_PROVIDER=fake            # fake | ollama | anthropic | openai
LLM_MODEL=qwen2.5:7b         # model name for the chosen provider
OLLAMA_BASE_URL=http://localhost:11434
ANTHROPIC_API_KEY=
OPENAI_API_KEY=
DATA_DIR=part1_data/data
CHROMA_DIR=runtime/chroma
APP_DB=runtime/app.db
```

7. **Check that it runs** (works once the skeleton is merged): `pytest` passes, and `uvicorn part5_web.app:app --reload` opens the website at http://127.0.0.1:8000.

Starting `requirements.txt` (the team lead adds it in week 1): `fastapi`, `uvicorn[standard]`, `jinja2`, `python-multipart`, `pydantic>=2`, `python-dotenv`, `chromadb`, `httpx`, `anthropic`, `openai`, `lxml`, `pytest`.

11. GitHub workflow

11.1 One-time setup (team lead)

1. Create a **private** repository named `regulatory-copilot` with a README and a Python `.gitignore`.
2. **Settings** → **Collaborators:** invite the four other members with *Write* access.
3. Push the folder skeleton from Section 7, `requirements.txt`, `.env.example`, and these guides in `docs/`.
4. **Settings** → **Branches** → **Add branch protection rule** for `main`: require a pull request before merging, require 1 approval, require review from Code Owners, and don't allow direct pushes.
5. Create `.github/CODEOWNERS` (replace the usernames):

```
/part1_data/        @member1
/part2_knowledge/   @member2
/part3_regulation/  @member3
/part4_support/     @member4
/part5_web/         @member5
/tests/integration/ @member4 @member5
/contracts/         @member1 @member2 @member3 @member4 @member5
```

6. Create `.github/pull_request_template.md` with these headings: *What this PR does · Which part · How I tested it · Screenshots (for website changes) · Contract changes? (yes/no).*
7. **Projects** → **New project** → **Board** with columns *To do*, *In progress*, *In review*, *Done*. Create labels `part-1` … `part-5`, `bug`, `question`, `contract`, `integration`.
8. Copy the issue list from each part guide (Section "GitHub issues to create") into Issues and assign them to their owners.

11.2 Daily routine (everyone)

```
git checkout main
git pull                                  # get everyone's latest work
git checkout -b part3/watcher-agent       # one branch per task
... write code + tests ...
pytest part3_regulation/tests             # your tests must pass
git add part3_regulation
git commit -m "feat(part3): add watcher agent with source-quote check"
git push -u origin part3/watcher-agent
```

Then on GitHub: **open a pull request** into `main`, fill the template, link the issue (write `Closes #12`), and request your reviewer. After approval, use **Squash and merge** and delete the branch.

11.3 Rules

- **Branch names:** `partN/short-description`, for example `part1/log-generator`, `part5/task-page`.

- **Commit messages:** `type(partN): what changed`, where type is `feat`, `fix`, `test`, `docs` or `refactor`.
- **Small pull requests.** Aim for under 400 changed lines. Open one at least every 3–4 days, so the others see your progress.
- **Only edit your own part folder.** Need a change in someone else's part? Open an Issue and assign them.
- **Reviewing is part of your grade.** A review must contain at least one real comment or question. Read the code, run it if you can, and check it follows the contracts.
- **Never commit:** `.env`, API keys, `.venv/`, `runtime/`, large files over 5 MB.
- **Pull `main` into your branch** (`git pull origin main`) before opening a PR, to catch conflicts early.

11.4 Fixing a merge conflict

1. Run `git pull origin main` on your branch. Git lists the conflicted files.
2. Open each file. Keep the right lines between the markers `<<<<<<<`, `=======` and `>>>>>>>`, then delete the markers.
3. Run the tests, then `git add <file>`, `git commit`, and `git push`.
4. If the conflict is in someone else's code, call them before choosing what to keep.

12. Building your part with your own AI tool

Each member may use any AI coding tool (Claude Code, Cursor, Copilot, ChatGPT and so on). The rules below keep five different tools producing one coherent system.

12.1 Rules

1. **Give the tool the context first:** this guide, your part guide, and the current `contracts/schemas.py`. Tell it which folders it may edit.
2. **Work in small steps:** one function or one page at a time, each with its tests. Never ask for "the whole part" at once.
3. **You must understand every line you merge.** In the final presentation, any member may be asked to explain any line of their part.
4. **The tool must not change other parts' folders or the contracts.** If it suggests that, stop and open an Issue.
5. **Never paste secrets** (API keys) into an AI tool or into code.
6. **Check AI-generated data** (texts, examples) for consistency before committing.
7. **Keep a short `AI_LOG.md` in your folder:** the main prompts you used and what you changed by hand. It counts as evidence of your work.

12.2 Starter prompt template

Each part guide contains a ready-made starter prompt. The general shape is:

```
You are helping me build Part N of a 5-person student project, the "Regulatory & Support
Copilot".
I attached: the project guide, my part guide, and contracts/schemas.py.
Rules: Python 3.11, Pydantic v2, follow the contracts exactly, edit only <my folders>,
write pytest tests for every function, keep functions small and commented, no new libraries
without asking me. First, read the documents and give me a short plan of the files you will
create. Wait for my approval before writing code. Then do task: <one task from my guide>.
```

13. Timeline

The plan assumes 8 weeks. If your deadline is different, keep the order and stretch or squeeze the weeks.

Week	Everyone	Milestone
1	Setup (Section 10), choose parts, repository skeleton, Contract Day	**M1:** contracts merged, stubs planned
2	Merge stubs for every interface; start real work; Part 1 delivers a first data sample	Everyone can import everyone's functions
3	Core building with the `fake` LLM; Part 2 search works on sample data	Each part has working code + tests
4	**First integration:** both workflows run end to end on the website with `fake` LLM	**M2:** clickable end-to-end prototype
5	Switch to a real LLM (Ollama or API); improve prompts and evidence	Real answers in both workflows
6	Quality week: fix bugs, handle errors, improve website and safety	Feature freeze at the end of the week
7	Evaluation run, final data fixes, report writing	**M3:** evaluation report ready
8	Rehearse the demo twice, polish slides, final report	**M4:** final presentation

Weekly meeting (30 minutes, same day every week): each member shows a 3-minute demo of what they merged, says what is next, and names any blocker. The team lead updates the board.

14. Integration and definition of done

14.1 Integration order

1. Parts 1 and 2 first: data, then ingestion and search.
2. Part 3 and Part 4 agents use `search()` and the tools, tested alone with the command line.
3. Part 4's orchestrator connects the agents; test with `python -m part4_support.orchestrator.run`.
4. Part 5's website calls the orchestrator; test both demos in the browser.
5. Safety (masking, audit) is connected throughout; evaluation runs last.

Integration tests live in `tests/integration/` and are written by Parts 4 and 5 together: one test per workflow that runs the whole chain with the `fake` provider.

14.2 The project is done when

- [ ] Demo A and Demo B (Section 3) work on a fresh clone by following README.md.
- [ ] Every agent output shows its sources; no change request or reply is "sent" without approval.
- [ ] Account numbers in tickets and logs are masked before any LLM call (tested).
- [ ] The audit log records every agent step and human decision.
- [ ] `pytest` passes; each part has tests for its public functions.
- [ ] The evaluation report exists and is shown on the website.
- [ ] Each folder has a README, and each member has an `AI_LOG.md`.
- [ ] The final report and slides are in `docs/`.

15. Knowing each other's work

Each member owns one part but must understand all five. These habits make that happen without extra meetings:

- **Review rotation** (Section 5) means you read one other part's code every week.
- **Weekly 3-minute demos** show everyone's progress.
- **Each folder README** explains what the part does, its inputs and outputs, and how to run it, in under one page.
- **Week 7 swap session (1 hour):** each member explains a teammate's part to the group, and the owner corrects them. If you cannot explain it, ask questions until you can.
- **Final presentation:** each member presents their part, but questions may go to anyone.

16. Final deliverables

Deliverable	Where	Owner
Working prototype (code)	GitHub repository	All
README with install and run steps	`README.md`	Part 5 (all contribute)
Evaluation report	`docs/evaluation-report.md` + Evaluation page	Part 5 (data from Part 1)
Final report (problem, design, results, lessons)	`docs/final-report.pdf`	All, one section each
Slides + live demo	`docs/slides.pdf`	All
Animated explainer (already made)	`presentation/copilot-explainer.html`	—

Suggested final report outline: 1 Introduction and problem · 2 Solution overview · 3 Architecture · 4 One section per part (written by its owner) · 5 Safety · 6 Evaluation results · 7 Limitations and future work · 8 What we learned · Appendix: AI tool usage.

17. Glossary

Term	Meaning
Agent	A Python function that uses an LLM plus tools to do one job and return a structured result
Orchestrator	The code that decides which agent runs next and keeps the task's state
LLM	Large Language Model, the "brain" that reads and writes text
RAG	Retrieval-Augmented Generation: search our documents first, then let the LLM answer from what was found
Embedding	A list of numbers that represents the meaning of a text; similar texts get similar numbers
Vector database	A database that finds texts by meaning (Chroma in our project)
Chunk	A small piece of a document (a section or a few paragraphs) stored in the vector database
ISO 20022	The international standard for payment messages (for example pacs.008 = customer payment, pacs.002 = payment status)
Reason code	A short code explaining why a payment was rejected (for example AC04 = closed account)
Circular	An official notice from a central bank announcing a new rule
Change request (CR)	A formal description of a change the product needs, with acceptance criteria and tests
Human-in-the-loop	A person must approve before an AI result is used
Contract	The agreed data shape or function signature that connects two parts
Stub	A placeholder function with the real signature that returns example data
Mock data	Realistic but invented data used instead of real company data